МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ» ННК «ІПСА»

Кафедра системного проектування

Курсова робота з дисципліни «Об'єктно-орієнтоване програмування»

Виконав студент
групи ДА-11
Сидько Нікіта
22 Варіант

Київ – 2023

Зміст

Завдання……...3
Засоби для реалізації……...3
Класи і ієрархія класів, їх реалізація…….5
Графіка…….9
Висновок……14
Література……..15

Завдання

Розробити програму з графічним інтерфейсом користувача на тему “Здоров'я”.

Завдання ми виконували командою з трьох людей, тому і поділили його на три частини :

1. Їжа
2. Вправи
3. Підрахунок спожитих/спалених калорій (щоденник)

Моя частина роботи впала на вправи. Згідно завдання я мав написати код, що буде робити підрахунок спалених калорій при виконанні одної з обраних користувачем вправ. У графічній частині коду мною також була оформлена ця частина.

Засоби для реалізації

Як мову програмування довго дискутуючі ми вибрали C++, а графічна частина реалізована на QT, бо це не складно у порівнянні з іншими (наприклад ImGUI або wxWidgets).

Qt є крос-платформовим фреймворком розробки програмного забезпечення, який дозволяє створювати широкий спектр додатків, включаючи графічні інтерфейси користувача, консольні програми, веб-додатки, мобільні додатки та інше. Основними особливостями Qt є наступні:

- Крос-платформовість: Qt підтримує розробку програм для різних операційних систем, включаючи Windows, macOS, Linux, Android та iOS. Це дозволяє розробникам писати один код, який працюватиме на різних платформах без необхідності внесення змін.

- Графічний інтерфейс користувача: Qt має потужні інструменти для створення графічного інтерфейсу користувача (GUI) з використанням набору віджетів. Віджети Qt дозволяють швидко і просто створювати кнопки, поля введення, таблиці, діалогові вікна та багато іншого. Сигнали та слоти: Qt використовує механізм сигналів та слотів для реалізації комунікації між об'єктами. Це дозволяє легко зв'язувати події (сигнали), які виникають у програмі, з відповідними обробниками (слотами), що спрощує розробку програм з подійною моделлю.

- Модульність та розширюваність: Qt складається з багатьох модулів, які можна використовувати поодинці або разом для створення потужних додатків. Фреймворк надає велику кількість функцій і класів для різних задач, таких як мережеве програмування, робота з базами даних, робота з графікою, паралельне програмування та інше.

- Інструменти розробки: Qt надає набір потужних інструментів для розробки, включаючи Qt Creator - інтегроване середовище розробки (IDE), яке має підтримку автодоповнення коду, налагодження, дизайнер графічного інтерфейсу та багато іншого. Велике співтовариство та документація: Qt має активне співтовариство розробників, яке надає підтримку, допомогу та відповіді на питання. Крім того, Qt має добре документовану офіційну документацію та розробницькі ресурси, що сприяють ефективному вивченню та використанню фреймворку.

Qt є потужним інструментом для розробки програмного забезпечення з великою функціональністю, широким спектром можливостей та підтримкою крос-платформовості. Він використовується в багатьох сферах, включаючи розробку десктопних додатків, мобільних додатків, вбудованих систем, ігор та багато іншого.

Класи і ієрархія класів, їх реалізація

Так як я відповідав за вправи, тут і далі, буде наведений код що відповідає за вправи.

Класс Exercice - основний клас, який буде успадковуватися в class EnduranceEx та Basic_Exercises (класи різних типів вправ). Клас передає методи, та змінні : повтори вправ, кількість разів (для виконання певних типів вправ)

```cpp
#pragma once
#include <string>

class Exercise
{
protected:
    double burned_cal = 0;
    int my_repeat = 0;
    std::string type_of_ex;

protected:
    Exercise() = default;
    void Set_burned_cal(double burned_cal);
    void Set_my_repeat(int my_repeat);
    virtual double Do() = 0;
    virtual double Coef_Calculate() = 0;

public:
    double Get_burned_cal() const;
    int Get_my_repeat() const;
    std::string Get_type_of_ex() const;
};
```

Реалізація методів

```cpp
#include "headers/Exercise.h"

void Exercise::Set_burned_cal(double burned_cal)
{
    this->burned_cal = burned_cal;
}

void Exercise::Set_my_repeat(int my_repeat)
{
    this->my_repeat = my_repeat;
}

double Exercise::Get_burned_cal() const
{
    return burned_cal;
}

int Exercise::Get_my_repeat() const
{
    return my_repeat;
}

std::string Exercise::Get_type_of_ex() const
{
    return type_of_ex;
}
```

Далі в нас ідуть два класи EnduranceEx та Basic_Exercises. Вони відповідають за типи вправ (різні методи підрахунку спалення калорій). У кожному класі наявний віртуальний метод Do (), який відповідає за виконання (підрахунок спалених калорій при виконанні вправи)

```h
#pragma once
#include "Exercise.h"

class Basic_Exercises : public Exercise
{
public:
    double Do() override;
    int Get_my_times() const;
    void Set_my_times(int my_times);

protected:
    double averagetimes_per_min;
    double average_burned;
    int my_times = 0;
};
```

```h
#pragma once
#include "Exercise.h"

class EnduranceEx : public Exercise
{
public:
    double Do() override;
    void Set_my_distance(int my_distance);
    int Get_my_distance() const;

protected:
    double averagetimes_per_min;
    double average_burned;
    int my_distance = 0;
};
```

Реалізація

```cpp
#include "headers/BasicEx.h"

double Basic_Exercises::Do()
{
    Set_my_repeat(my_repeat);
    Set_my_times(my_times);
    double coef = Coef_Calculate();
    double b_cal = my_times * my_repeat * coef;
    Set_burned_cal(b_cal);
    return b_cal;
}

void Basic_Exercises::Set_my_times(int my_times)
{
    this->my_times = my_times;
}

int Basic_Exercises::Get_my_times() const
{
    return my_times;
}
```

```cpp
#include "headers/EnduranceEx.h"

double EnduranceEx::Do()
{
    int dist = my_distance * my_repeat;
    Set_my_distance(dist);
    double coef = Coef_Calculate();
    double b_cal = dist * coef;
    Set_burned_cal(b_cal);
    return b_cal;
}

void EnduranceEx::Set_my_distance(int my_distance)
{
    this->my_distance = my_distance;
}

int EnduranceEx::Get_my_distance() const
{
    return my_distance;
}
```

І нарешті ми доходимо до самих вправ. Тут я намагався реалізувати гоф патерн “стратегія”. Стратегія - поведінковий патерн проектування, який визначає сімейство подібних алгоритмів і поміщає кожен із новачків у свій

клас, після чого алгоритми можна взаємозамінювати безпосередньо під час виконання програми. Наприклад у нас є клас Basic_Ex, методи якого – певні вправи (PushUp, PullUp …) Для кожної вправи ми рахуємо певний коефіцієнт, який будемо множити на кількість виконаних разів.

Тобто, маємо наступне:

```
class Basic_Exercises
{
public:
    int coef_for_exercise;
    int burned_cal;
    std::string type_of_ex;

    int Do ();
    int PushUp ();
    int PullUp ();
    int Coef_Calculate();
};
```

Тобто, для кожного методу (вправи) ми рахуємо свій коефіцієнт (Coef_calculate ()) і виконуємо цю вправу (Do ()) , отримуючи спалені калорії.

```
int Basic_Exercises::Coef_Calculate()
{
    if (type_of_ex == "Push_up")
    {
        int a,b;
        coef_for_exercise = a * b;
        return coef_for_exercise;
    }
    if (type_of_ex == "Pull_up")
    {
        int c, d;
        coef_for_exercise = c*d;
        return coef_for_exercise;
    }
}
int Basic_Exercises::PushUp()
{
    Coef_Calculate();
}
int Basic_Exercises::PullUp()
{
    Coef_Calculate();
}

int Basic_Exercises::Do()
{
    if (type_of_ex == "Push_up")
    {
        return burned_cal;
    }
    if (type_of_ex == "Push_up")
    {
        return burned_cal;
    }
}
```

В нас виникає проблема, кожен раз, коли ми додаємо якийсь новий метод, ми постійно будемо змінювати функцію coef_calcuate та do. Вирішує цю проблему патерн “Стратегія” який я намагався реалізувати у роботі , а саме,

кожен метод буде записаний як окремий клас у якому буде перевизначатися метод coef_calcuate, а метод do можна зробити спільним для всіх вправ. (слід зазначити що у лабі методи Do () та Coef_Calculate() віртуальні і обьявлені у класі ексерсайз (було потрібно для лаби))

Type_Basic_Exercises.h

```cpp
#pragma once
#include "BasicEx.h"
#include <cmath>

class Twisting : public Basic_Exercises
{
public:
    Twisting(int My_repeat, int My_times);
private:
    double Coef_Calculate() override;
};

class Push_Up : public Basic_Exercises
{
public:
    Push_Up(int My_repeat, int My_times);
private:
    double Coef_Calculate() override;
};

class Pull_Up : public Basic_Exercises
{
public:
    Pull_Up(int My_repeat, int My_times);
private:
    double Coef_Calculate() override;
};

class Squats : public Basic_Exercises
{
public:
    Squats(int My_repeat, int My_times);
private:
    double Coef_Calculate() override;
};
```

Type_Endurance_Exercises.h

```cpp
#pragma once
#include "EnduranceEx.h"
#include <cmath>

class Run : public EnduranceEx
{
public:
    Run(int My_repeat, int My_distance);
protected:
    double Coef_Calculate() override;
};
class Bicycle : public EnduranceEx
{
public:
    Bicycle(int My_repeat, int My_distance);
protected:
    double Coef_Calculate() override;
};
```

Реалізація Type_Basic_Exercises.h

```cpp
#include "headers/Type_Basic_Exercises.h"

Twisting::Twisting(int My_repeat, int My_times)
{
    my_repeat = My_repeat;
    my_times = My_times;
    this->averagetimes_per_min = 40;
    this->average_burned = 5;
    this->type_of_ex = "Twisting";
}

double Twisting::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min) * 14;
    coef = round(coef * 10) / 10;
    return coef;
}
```

```cpp
#include "headers/Type_Basic_Exercises.h"

Squats::Squats(int My_repeat, int My_times)
{
    my_repeat = My_repeat;
    my_times = My_times;
    this->averagetimes_per_min = 45;
    this->average_burned = 13.5;
    this->type_of_ex = "Squats";
}

double Squats::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min) * 13;
    coef = round(coef * 10) / 10;
    return coef;
}
```

```cpp
#include "headers/Type_Basic_Exercises.h"

Push_Up::Push_Up(int My_repeat, int My_times)
{

    my_repeat = My_repeat;
    my_times = My_times;
    this->averagetimes_per_min = 30;
    this->average_burned = 4;
    this->type_of_ex = "Push_Up";
}

double Push_Up::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min) * 10;
    coef = round(coef * 10) / 10;
    return coef;
}
```

```cpp
#include "headers/Type_Basic_Exercises.h"

Pull_Up::Pull_Up(int My_repeat, int My_times)
{
    my_repeat = My_repeat;
    my_times = My_times;
    this->averagetimes_per_min = 10;
    this->average_burned = 2;
    this->type_of_ex = "Pull_Up";
}

double Pull_Up::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min) * 5;
    coef = round(coef * 10) / 10;
    return coef;
}
```

Реалізація Type_Endurance_Exercises.h

```cpp
#include "headers/Type_Endurance_Exercises.h"

Run::Run(int My_repeat, int My_distance)
{
    my_repeat = My_repeat;
    my_distance = My_distance;
    this->averagetimes_per_min = 170;
    this->average_burned = 14;
    this->type_of_ex = "Run";
}
double Run::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min);
    coef = round(coef * 100) / 100;
    return coef;
}

```

```cpp
#include "headers/Type_Endurance_Exercises.h"

Bicycle::Bicycle(int My_repeat, int My_distance)
{
    my_repeat = My_repeat;
    my_distance = My_distance;
    this->averagetimes_per_min = 340;
    this->average_burned = 10;
    this->type_of_ex = "Bicycle";
}
double Bicycle::Coef_Calculate()
{
    double coef = (average_burned / averagetimes_per_min);
    coef = round(coef * 100) / 100;
    return coef;
}
```

Графіка

Вся графіка виконана за допомогою фреймворку QT. Маючи частину завдання – вправи, мною було реалізовано Add Exercises.

addexercise.h

```cpp
#ifndef ADDEXERCISE_H
#define ADDEXERCISE_H

#include <QDialog>
#include "headers/Diary.h"

namespace Ui // використовується в коді пов'язаний з інтерфейсом користувача (UI) за допомогою Qt.
{
    class AddExercise; //  клас, який представляє вікно додавання вправ у інтерфейсі користувача.
}
/*
 * клас AddExercise є підкласом класу QDialog і успадковує всі його властивості та функціональність.
 * QDialog є класом у фреймворку Qt, який представляє діалогове вікно. Діалогове вікно є модальним вікном,
 * яке використовується для взаємодії з користувачем та отримання даних.
*/
class AddExercise : public QDialog
{
    Q_OBJECT // макрос у Qt, який вказує на те, що клас використовує функціональність подій Qt.

public:
    explicit AddExercise(QWidget *parent = nullptr);// конструктор
    ~AddExercise(); // деструктор

    std::size_t getTimes(); // повертає кількість разів (вик врава)
    std::size_t getReps();  // повертає кількість повторів (вик вправа)

// оголошення трьох приватних слотів
private slots:
    void on_FoodCancelButton_clicked();
```

```
    void on_comboBox_activated(int index);

    void on_FoodAddButton_clicked();

// оголошує список сигналів, які можуть бути імітовані класом
signals:
    void sendBicycle(const Bicycle& bicycle);
    void sendRun(const Run& run);
    void sendPull_Up(const Pull_Up& bicycle);
    void sendPush_Up(const Push_Up& bicycle);
    void sendTwisting(const Twisting& bicycle);
    void sendSquats(const Squats& bicycle);

private:
    Ui::AddExercise *ui;
    int index_;
};

#endif // ADDEXERCISE_H
```

Реалізація :

```
#include "addexercise.h"
#include "ui_addexercise.h"
#include <QMessageBox>

AddExercise::AddExercise(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::AddExercise)
{
    ui->setupUi(this);
}

AddExercise::~AddExercise()
{
    delete ui;
}

size_t AddExercise::getTimes()
{
    bool flag;
    std::size_t size = ui->TimeslineEdit->text().split(" ")[0].toUInt(&flag);
    if(!flag)
    {
        QMessageBox::warning(this, "Attention","Times must be a number!");
        return 0;
    }
    return size;
}

size_t AddExercise::getReps()
{
    bool flag;
    std::size_t size = ui->RepslineEdit->text().split(" ")[0].toUInt(&flag);
    if(!flag)
    {
        QMessageBox::warning(this, "Attention","Reps must be a number!");
        return 0;
    }
    return size;
}

```

```cpp
void AddExercise::on_FoodCancelButton_clicked()
{
    close();
}


void AddExercise::on_comboBox_activated(int index)
{
    index_ = index;
}


void AddExercise::on_FoodAddButton_clicked()
{
    /*
     *генерується сигнал sendBicycle з параметром bicycle.
     * Цей сигнал може бути підключений (з'єднаний) до слотів (функцій), які приймають аргумент типу Bicycle,
     *  і коли сигнал генерується, відповідні слоти будуть викликані з переданим значенням bicycle.
     */
    if(index_ == 0)
    {
        Bicycle bicycle(getTimes(), getReps());
        emit sendBicycle(bicycle);
    }
    if(index_ == 1)
    {
        Run run(getTimes(), getReps());
        emit sendRun(run);
    }
    if(index_ == 2)
    {
        Pull_Up pull_Up(getTimes(), getReps());
        emit sendPull_Up(pull_Up);
    }
    if(index_ == 3)
    {
        Push_Up push_Up(getTimes(), getReps());
        emit sendPush_Up(push_Up);
    }
    if(index_ == 4)
    {
        Twisting twisting(getTimes(), getReps());
        emit sendTwisting(twisting);
    }
    if(index_ == 5)
    {
        Squats squats(getTimes(), getReps());
        emit sendSquats(squats);
    }
    close();
}
```

foodandexercise.h

```cpp
#ifndef FOODANDEXERCISES_H
#define FOODANDEXERCISES_H

#include <QDialog>
#include "headers/Diary.h"

namespace Ui {
class FoodAndExercises;
}

class FoodAndExercises : public QDialog
{
    Q_OBJECT

public:
    explicit FoodAndExercises(QWidget *parent = nullptr);
    ~FoodAndExercises();

public slots:
    void getFood(const std::vector<Food>& food);
    void getExercises(const std::map<std::string, std::pair<std::size_t, std::size_t>>& exercises);

private slots:
    void on_CloseButton_clicked();

private:
    Ui::FoodAndExercises *ui;
};

#endif // FOODANDEXERCISES_H
```

foodandexercise.cpp (частина вправ)

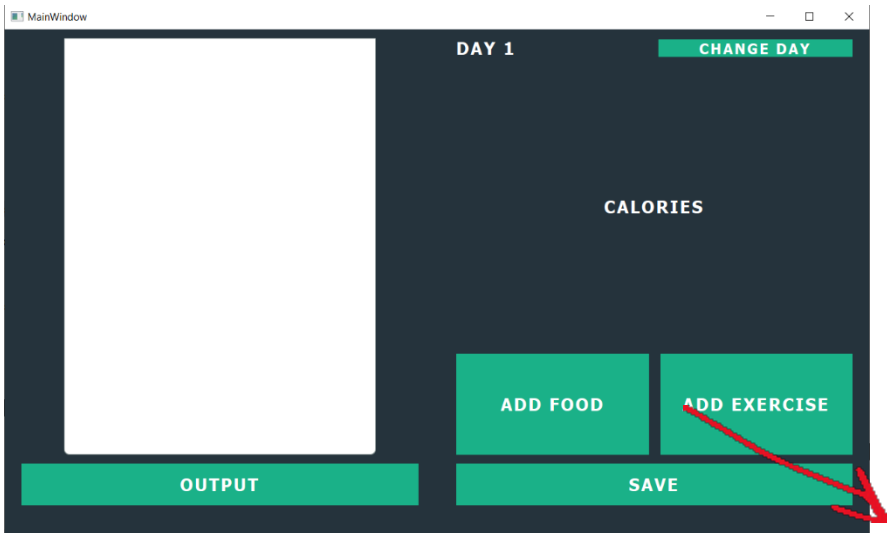
```cpp
FoodAndExercises::FoodAndExercises(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::FoodAndExercises)
{
    ui->setupUi(this);
    ui->FoodTableWidget->setColumnCount(6);
    ui->FoodTableWidget->setHorizontalHeaderLabels({"Name", "Weight", "Calories", "Fats", "Carbhydr", "Proteins"});

    ui->ExerciseTableWidget->setColumnCount(3);
    ui->ExerciseTableWidget->setHorizontalHeaderLabels({"Name", "Times", "Reps"});
}
```

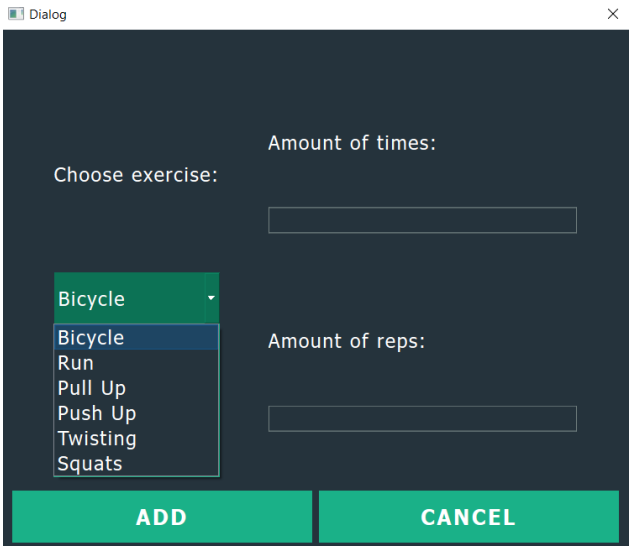
```cpp
void FoodAndExercises::getExercises(const std::map<string, std::pair<size_t, size_t> > &exercises)
{
    for(const auto& el : exercises)
    {
        auto rowCount = ui->ExerciseTableWidget->rowCount();
        ui->ExerciseTableWidget->insertRow(rowCount);
        ui->ExerciseTableWidget->setItem(rowCount,0, new QTableWidgetItem(el.first.c_str()));
        ui->ExerciseTableWidget->setItem(rowCount,1, new QTableWidgetItem(std::to_string((std::size_t)el.second.first).c_str()));
        ui->ExerciseTableWidget->setItem(rowCount,2, new QTableWidgetItem(std::to_string((std::size_t)el.second.second).c_str()));
    }
}

void FoodAndExercises::on_CloseButton_clicked()
{
    close();
}
```

Головне меню програми :

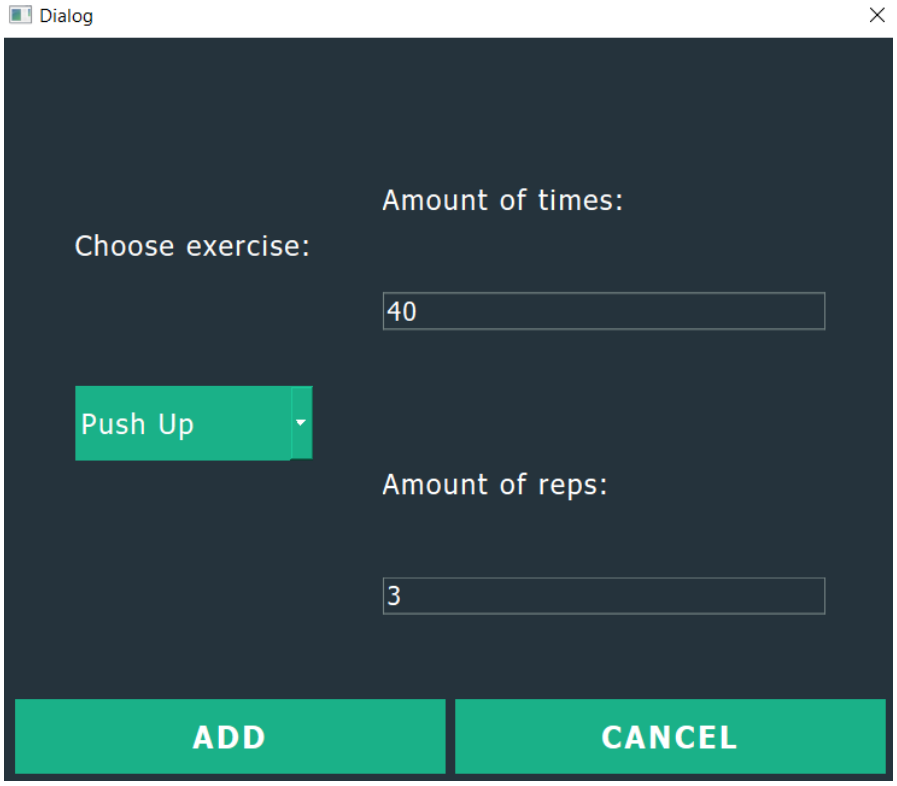


Вправи

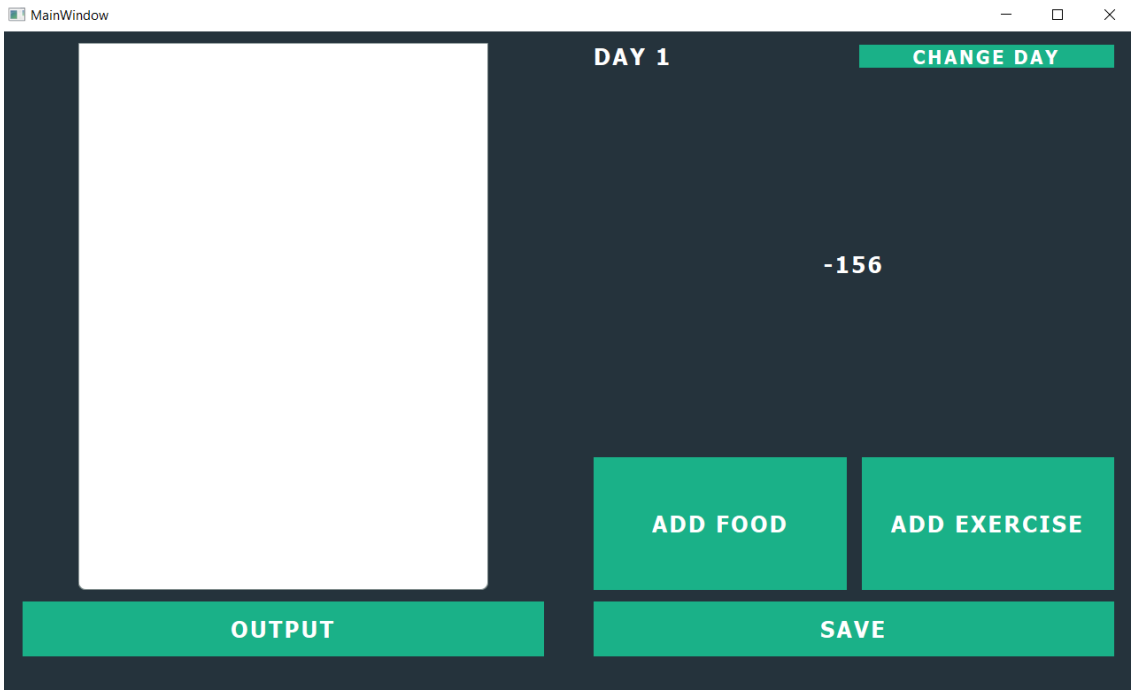


Бачимо, що в нас є можливість обрати вправу, та записати кількості разів виконання та повторів.

Оберемо вправу, кількість виконуваних разів та кількість повторів.



У головному меню бачимо, що в нас спалилися калорії :



Висновок.

У ході виконання курсової роботи було зроблено програму підрахунку калорій. Користувач має доступ до налаштування вправ що в подальшому буде виконувати.

При написані роботи було використано мову програмування C++ та фреймворк QT. Для побудови графічного інтерфейсу були використані різні компоненти, а для реагування на дії користувача, такі як вибір відповіді або натискання кнопок, були використані обробники подій.

Окрім того, програма дозволяє вести облік спожитих та спалених калорій за день і зберігати ці дані у текстовий файл. Це допомагає користувачеві зберігати та аналізувати свої харчові та фізичні звички.

Література

- https://habr.com/ru/articles/501798/
- https://doc.qt.io/qt-6/designer-creating-custom-widgets.html
- https://doc.qt.io/
- stackoverflow.com